> **NOTE** Interestingly, in the limit where , the point that maximizes UCB's acquisition score is where the standard deviation is maximized, that is, where our uncertainty is at its highest. This behavior corresponds to pure exploration, as we are selecting data points with high uncertainty.

Finally, UCB correctly assigns higher scores to data points that offer better balance between exploration and exploitation.

- If two data points have the same predictive mean but different predictive standard deviations, then the one with the higher uncertainty will have a higher score. The policy therefore rewards exploration.
- If two data points have the same predictive standard deviations but different predictive means, then the one with the higher mean with have a higher score. The policy therefore rewards exploitation.

Remember that one of the policies discussed in Chapter 4, *Expected Improvement*, also has this property, which is a desideratum of any BayesOpt policy.

All in all, this parameter β controls UCB and how the policy explores and exploits the search space. This means that by setting the value of this parameter, we have direct control over its behavior. Unfortunately, beyond the fact that the value for β corresponds to the level of exploration, there's not a straightforward, principled way of setting this parameter, and some value might work on some problems but not others.

> **NOTE** **Default values for the parameter**
>
> The documentation of BoTorch usually shows UCB with = 0.1, while many research papers that use UCB for BayesOpt opt for = 3. So, 0.1 could be your go-to value when using this policy if you prefer exploitation, and 3 should be the default if you lean towards exploration.

Exercise 1 of this chapter further discusses a more involved method of setting β that might generally work sufficiently well.

5.2.3 Implementation with BoTorch

Having thoroughly discusses the motivation and math behind UCB, we will now learn how to implement the policy using BoTorch. The code we will be looking at is included in `CH05/01 - BayesOpt loop.ipynb`. Remember from Section 4.2.2 that although we could manually implement the *Probability of Improvement* policy ourselves, declaring the BoTorch policy object, using it with the GP model, and optimizing the acquisition score using BoTorch's helper function `optimize_acqf()` make it easier to implement our BayesOpt loop.

©Manning Publications Co. To comment go to liveBook
https://livebook.manning.com/#!/book/bayesian-optimization-in-action/discussion

Licensed to ashwin sampath <ashwin.manning@gmail.com>